KIROVAULT

Security, Privacy & Permissions

Security requirements for a local-first file manager that handles user-controlled paths, previews and destructive operations.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Threat model

• Untrusted inputs include paths, filenames, file contents, metadata, archive members and corrupted media.

• Threats include path traversal, unintended destructive operations, renderer vulnerabilities, resource exhaustion and privacy leakage.

• Assume files can change outside KiroVault between indexing and use; revalidate at action time.

Least privilege

• Use narrow Tauri capabilities and task-specific commands; avoid blanket filesystem access.

• Request access only to locations required for the user's action.

• Never request administrator privileges automatically to bypass denied access.

• Use Rust-side path validation and explicit operation scopes.

Preview hardening

• Render text as escaped text; sanitize/disable active HTML and remote resources.

• Never run scripts, Office macros or executable content on selection or preview.

• Apply timeouts, size limits and bounded worker queues to decoders.

• Do not silently install fonts or extract archives.

Destructive action safety

• Use Recycle Bin where available and make permanent deletion explicit.

• Confirm bulk/permanent deletes and potentially destructive conflict choices.

• Show source, destination and operation scope before ambiguous moves.

• Return per-item results; never claim full success if some items failed.

Privacy controls

• Local index/cache by default; no cloud account required for core tasks.

• No default collection or upload of filenames, paths, file contents or library activity.

• Make indexing roots, exclusions, cache location and content indexing visible and configurable.

• Diagnostic exports should be user-initiated and reviewed before sharing.

Security verification

• Test traversal attempts, junctions/symlinks, path collisions and permission boundaries.

• Test malicious-looking HTML/text, oversized files, corrupt media and archive member paths.

• Review third-party licenses and bundled media components before distribution.

• Keep dependencies updated and document the threat model and residual risks.

References

- Tauri security overview: https://v2.tauri.app/security/
- Tauri capabilities: https://v2.tauri.app/security/capabilities/
- Microsoft file security and access rights: https://learn.microsoft.com/en-us/windows/win32/fileio/file-security-and-access-rights
- FFmpeg legal notes: https://ffmpeg.org/legal.html

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.